Name of Student : Raghav Mahesh Sarda	**PRN:** 25070122***235***
Subject: Programming with Java	**Semester: III**
ASSIGNMENT No : 14	**Year: AY 2026-27**
TITLE : Exception Handling using Try-Catch	

Problem Statement

Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.

Learning Objectives

To handle runtime exceptions using try-catch, nested try, and multiple catch blocks.

Program:

MultiCatch.java

```
// Assignment 14 - Exercise 1: Exception handling with Multiple Catch
Blocks
import java.util.*;
public class MultiCatch
{
   public static void main(String args[]) // Main method
   {
        Scanner sc = new Scanner(System.in); // Scanner object
        int arr[] = {10, 20, 30, 40, 50}; // Initializing sample array
        System.out.println("Sample Array:");
        for(int i=0; i<5; i++)
        {
            System.out.print(arr[i]+" ");
        }
        System.out.println("\n");
        try
        {
            System.out.print("Enter index to access (0-4): ");
            int idx = sc.nextInt(); // Reading index
            System.out.println("Array value at index " + idx + " is: " +
arr[idx]); // Accessing array element
```

```
            System.out.print("Enter divisor for the value: ");
            int div = sc.nextInt(); // Reading divisor
            int res = arr[idx] / div; // Arithmetic division
            System.out.println("Division Result = " + res); // Printing
quotient
        }
        catch(ArrayIndexOutOfBoundsException e) // Catching invalid index
        {
            System.out.println("Error: Index entered is beyond array
limits!"); // Handling array exception
        }
        catch(ArithmeticException e) // Catching division by zero
        {
            System.out.println("Error: Division by zero is not
allowed!"); // Handling math exception
        }
        catch(Exception e) // Generic exception fallback
        {
            System.out.println("Error: Exception occurred! " +
e.getMessage()); // Generic message
        }
        System.out.println("Program execution continues normally.");
        sc.close();
        System.out.println("\n");
    }
}
```

NestedTryCatch.java

```
// Assignment 14 - Exercise 2: Exception handling with Nested Try-Catch
Blocks
import java.util.*;

public class NestedTryCatch
{
    public static void main(String args[]) // Main method
    {
        Scanner sc = new Scanner(System.in); // Scanner object
        int nums[] = {100, 200, 300}; // Sample array
        System.out.println("Sample Array:");
        for(int i=0; i<3; i++)
        {
            System.out.print(nums[i]+" ");
        }
```

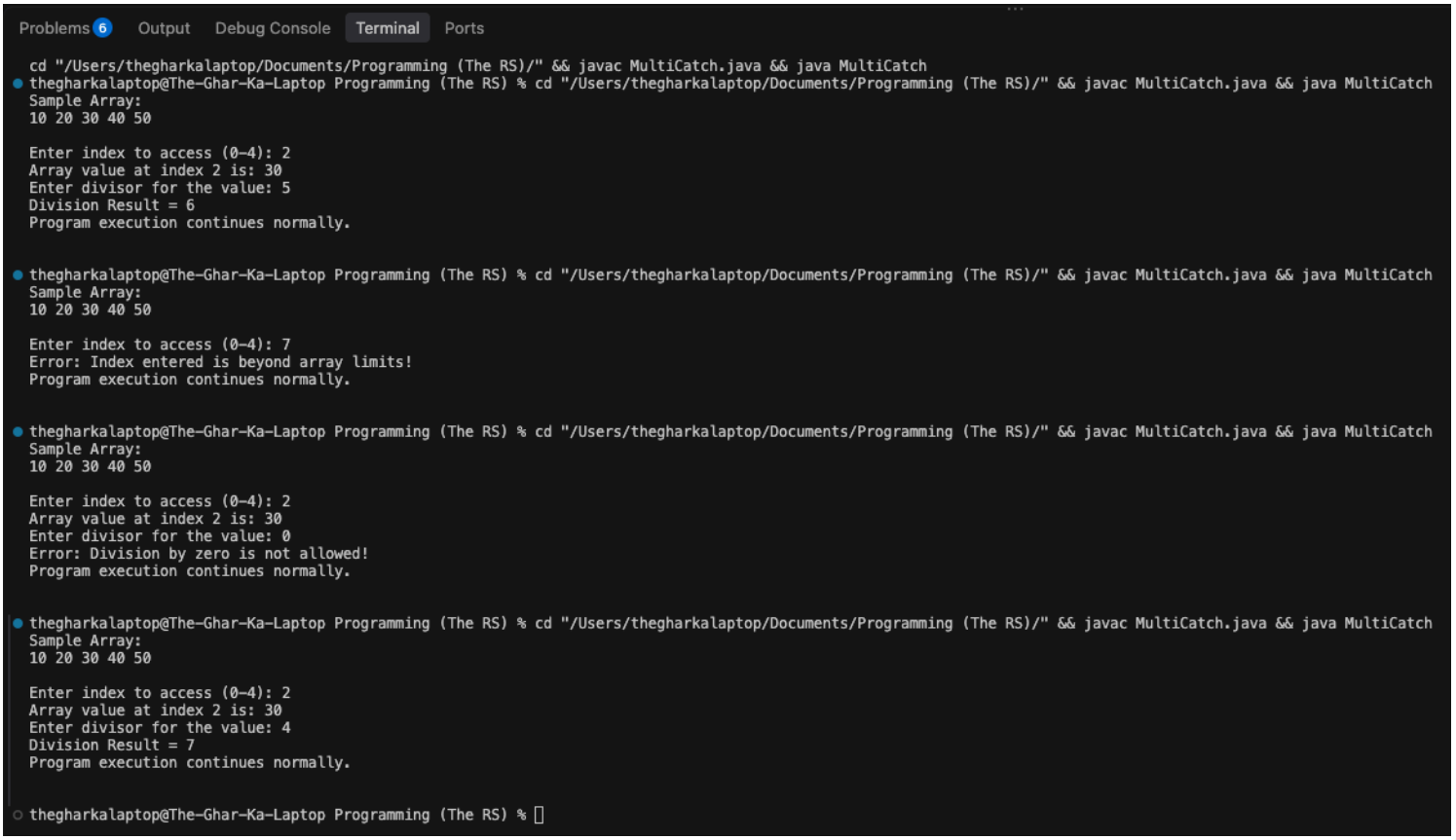
```
        System.out.println("\n");

        try // Outer Block
        {
            System.out.print("Enter array index to divide: ");
            int index = sc.nextInt(); // Reading index
            int val = nums[index]; // Potentially throws
ArrayIndexOutOfBoundsException

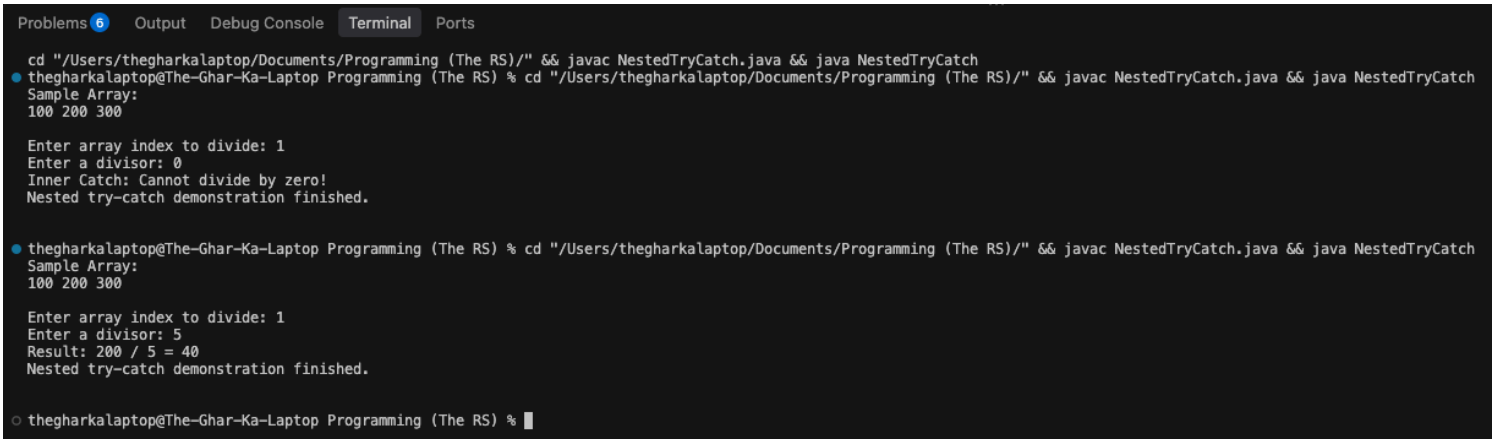
            try // Inner Block
            {
                System.out.print("Enter a divisor: ");
                int divisor = sc.nextInt(); // Reading divisor
                int quotient = val / divisor; // Potentially throws
ArithmeticException
                System.out.println("Result: " + val + " / " + divisor + "
= " + quotient); // Printing result
            }
            catch(ArithmeticException e) // Handling inner try block
exception
            {
                System.out.println("Inner Catch: Cannot divide by
zero!"); // Inner catch error
            }
        }
        catch(ArrayIndexOutOfBoundsException e) // Handling outer try
block exception
        {
            System.out.println("Outer Catch: Selected index does not
exist in array!"); // Outer catch error
        }
        System.out.println("Nested try-catch demonstration finished.");
// Completion message
        sc.close();
        System.out.println("\n");
    }
}
```

Output:

MultiCatch.java



NestedTryCatch.java



Conclusion:

In this assignment, the concepts of runtime error handling using single, multiple, and nested try-catch blocks were successfully implemented and verified. Multiple catch blocks ensured specific error messages were triggered for distinct runtime faults like division by zero and invalid array indexing, while nested try-catch structures enabled localized exception resolution without interrupting outer code flow.